Bài 1.9 – Toán: đạo hàm, gradient, Gradient Descent

> **Giai đoạn 1 – Python cho ML & Data Science** · Thời lượng: 2 buổi · Hình thức mở đầu: *Câu chuyện*

🎯 Sau bài này bạn sẽ

- Hiểu đạo hàm là "độ dốc", và tính được đạo hàm của những hàm ML hay dùng.
- Hiểu gradient là "la bàn chỉ hướng dốc nhất".
- Tự tay chạy **Gradient Descent**, thuật toán giúp máy "học", trên giấy và bằng code.
- Biết chọn **learning rate**: quá nhỏ thì chậm, quá lớn thì "nổ".
- Tự viết từ đầu một mô hình Linear Regression học từ dữ liệu.

Chia buổi:

- **Buổi 1:** Mục 1 → 4 (đạo hàm, đạo hàm riêng, gradient, quy tắc chuỗi).
- **Buổi 2:** Mục 5 → 9 (Gradient Descent, learning rate, tự viết mô hình học).

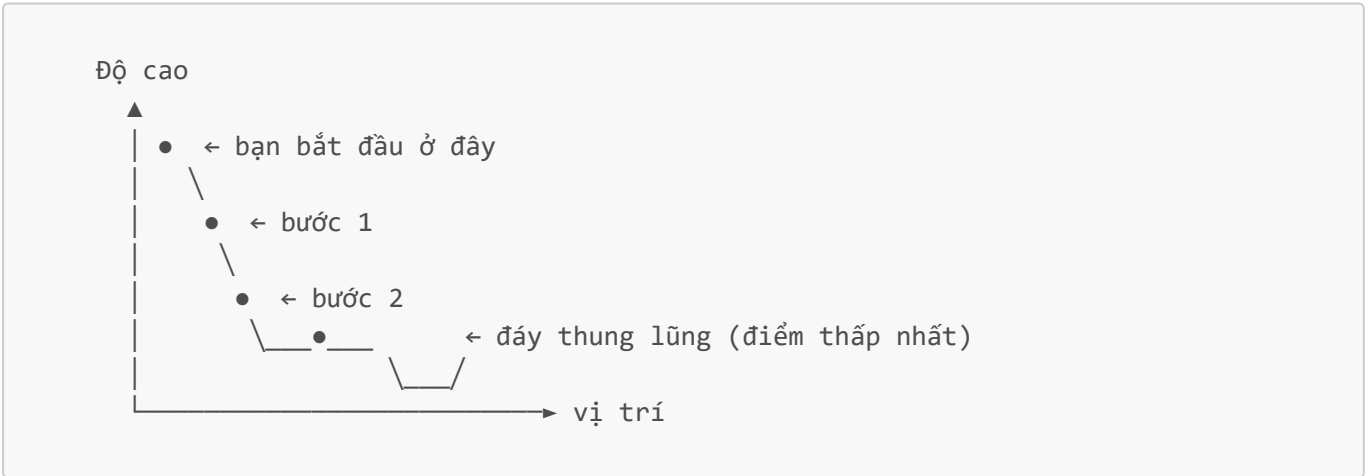
---

1. Câu chuyện: Xuống núi trong sương mù

Bạn bị lạc trên một sườn núi lúc chiều tối. Sương mù dày đặc, chỉ nhìn thấy được **một mét quanh chân**. Bạn muốn xuống thung lũng (nơi thấp nhất) để tìm đường về.

Bạn làm gì?

1. Dùng chân **dò xem hướng nào dốc xuống nhất**.
2. **Bước một bước** theo hướng đó.
3. Lặp lại, cho đến khi mọi hướng đều không còn dốc xuống nữa → bạn đã ở đáy.



Đó **chính xác** là cách máy học:

- "Độ cao" = **mức độ sai** của mô hình (gọi là **loss**).
- "Vị trí" = các **tham số** của mô hình (w, b ở bài 1.8).
- "Dò độ dốc" = tính **đạo hàm / gradient**.
- "Bước xuống" = cập nhật tham số.

Thuật toán này tên là **Gradient Descent (Hạ gradient)**. Nó đứng sau gần như mọi mô hình ML hiện đại, kể cả ChatGPT.

---

2. Vấn đề cụ thể: tìm w cho công thức giá nhà

Ở bài 1.8, ta **tự đặt** trọng số w. Giờ ta muốn **máy tự tìm** w tốt nhất từ dữ liệu.

Dữ liệu 5 căn hộ (bài 1.7), diện tích tính theo **chục m^2** cho số gọn:

Căn	x (chục m^2)	y (giá thật, tỷ)
1	4	2,0
2	5	2,6
3	6	2,9
4	7	3,6
5	8	4,0

Mô hình: **$\hat{y} = w \cdot x + b$**. Câu hỏi: w và b bằng bao nhiêu thì dự đoán **khớp nhất** với giá thật?

2.1. Đo "mức độ sai": hàm loss

- **Loss function (Hàm mất mát):** Một con số đo mô hình đang sai nhiều hay ít. Càng nhỏ càng tốt. (Sẽ học kỹ ở bài 2.2.)
- Ở đây dùng **MSE – Mean Squared Error (Sai số bình phương trung bình)**:

$$MSE = \frac{1}{n} \sum_{i=1}^{n} (\hat{y}_i - y_i)^2$$

Ký hiệu	Ý nghĩa
$\hat{y}_i - y_i$	Sai số của căn thứ i (dự đoán trừ thật)
$(...)^2$	Bình phương: sai âm hay dương đều tính là sai, sai nhiều bị phạt rất nặng
$1/n \, \Sigma$	Lấy trung bình trên n căn

Tính thử với w = 0, b = 0 (mô hình "ngây thơ", đoán mọi căn giá 0): sai số là −2; −2,6; −2,9; −3,6; −4. Bình phương: 4; 6,76; 8,41; 12,96; 16 → tổng 48,13 → MSE = 48,13 / 5 = **9,626**. Sai rất nhiều.

2.2. Vì sao không thử hết mọi giá trị?

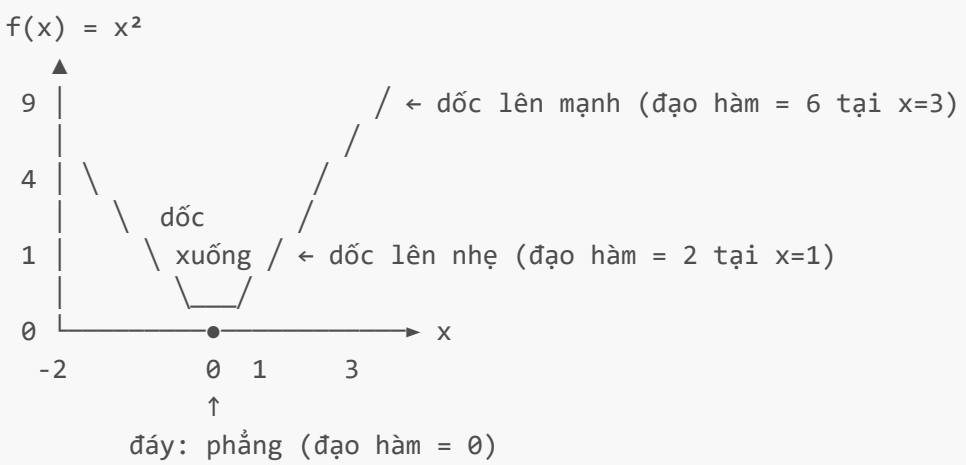
Thử w từ −10 đến 10, bước 0,01 → 2.000 giá trị. Thêm b → 2.000 × 2.000 = 4 triệu lần thử. Một mạng nơ-ron có **hàng triệu** tham số → số lần thử nhiều hơn số nguyên tử trong vũ trụ. Bất khả thi. Ta cần cách thông minh hơn: **đi theo độ dốc**.

---

3. Đạo hàm: độ dốc tại một điểm

3.1. Trực giác

- **Derivative (Đạo hàm):** Cho biết **khi x tăng thêm một chút xíu, f(x) thay đổi bao nhiêu** (và theo hướng nào).
- **Ví dụ đời thực:** Đồng hồ tốc độ xe máy. Quãng đường là hàm của thời gian. Đồng hồ hiện 40 km/h = "đạo hàm của quãng đường theo thời gian" tại khoảnh khắc đó.



Ba điều cần nhớ:

- Đạo hàm **dương** → hàm đang **đi lên** (sang phải thì cao hơn).
- Đạo hàm **âm** → hàm đang **đi xuống**.
- Đạo hàm **bằng 0** → chỗ **phẳng**, có thể là đáy.

3.2. Công thức "dò bằng chân"

$$f'(x) \approx \frac{f(x+h) - f(x)}{h} \qquad \text{với } h \text{ rất nhỏ (ví dụ } 0.00001)$$

Ký hiệu	Ý nghĩa
f'(x) (đọc "f phẩy")	Đạo hàm của f tại x
h	Một bước rất nhỏ
Tử số	f thay đổi bao nhiêu
Chia cho h	Thay đổi trên mỗi đơn vị x = độ dốc

Tính thử $f(x) = x^2$ tại x = 3, h = 0,001: $(3{,}001^2 - 3^2) / 0{,}001 = (9{,}006001 - 9) / 0{,}001 = 6{,}001 \approx \mathbf{6}$.

```python
def f(x):
    return x ** 2
```

```
def dao_ham_so(f, x, h=1e-5):          # 1e-5 = 0.00001
    return (f(x + h) - f(x)) / h       # "dò" độ dốc bằng một bước rất nhỏ

for x in [-2, 0, 1, 3]:
    print(f"x = {x:>2}: đạo hàm ≈ {dao_ham_so(f, x):.4f} | công thức 2x = {2 *
x}")
```

Kết quả mẫu:

```
x = -2: đạo hàm ≈ -4.0000 | công thức 2x = -4
x =  0: đạo hàm ≈ 0.0000 | công thức 2x = 0
x =  1: đạo hàm ≈ 2.0000 | công thức 2x = 2
x =  3: đạo hàm ≈ 6.0000 | công thức 2x = 6
```

3.3. Ba quy tắc đủ dùng cho bài này

Hàm	Đạo hàm	Ví dụ
Hằng số c	0	(5)' = 0
x^n	$n \cdot x^{n-1}$	$(x^2)' = 2x$; $(x^3)' = 3x^2$
$a \cdot f(x) + g(x)$	$a \cdot f'(x) + g'(x)$	$(3x^2 + 5x)' = 6x + 5$

Bạn **không cần** thuộc bảng đạo hàm dài. Thư viện (PyTorch) sẽ tự tính đạo hàm cho bạn (bài 3.8). Bạn chỉ cần hiểu **ý nghĩa**.

---

4. Nhiều biến: đạo hàm riêng, gradient, quy tắc chuỗi

4.1. Partial derivative (Đạo hàm riêng)

Loss phụ thuộc **hai** tham số w và b. Ta hỏi riêng từng cái:

- "Nếu **chỉ** tăng w một chút (giữ b nguyên), loss thay đổi bao nhiêu?" → $\partial L/\partial w$
- "Nếu **chỉ** tăng b một chút (giữ w nguyên)?" → $\partial L/\partial b$

Ký hiệu ∂ (đọc "đạo hàm riêng" hoặc "del") thay cho d.

Ví dụ đời thực: Giá một cốc trà sữa phụ thuộc lượng trà và lượng trân châu. "Đạo hàm riêng theo trân châu" = thêm 1 gram trân châu thì giá tăng bao nhiêu, **giữ nguyên lượng trà**.

Tính thử: $g(w, b) = w^2 + 3b$. Tại (w=2, b=1):

- $\partial g/\partial w$: coi b là hằng số → đạo hàm của w^2 là $2w$ = **4**.
- $\partial g/\partial b$: coi w là hằng số → đạo hàm của $3b$ là **3**.

```
def g2(w, b):
    return w ** 2 + 3 * b

h = 1e-5
w0, b0 = 2.0, 1.0
print(round((g2(w0 + h, b0) - g2(w0, b0)) / h, 3),    # chỉ nhích w
      round((g2(w0, b0 + h) - g2(w0, b0)) / h, 3))    # chỉ nhích b
```

Kết quả mẫu:

```
4.0 3.0
```

4.2. Gradient

- **Gradient:** Vector gom tất cả đạo hàm riêng lại: ∇L = [∂L/∂w, ∂L/∂b]. (∇ đọc là "nabla".)
- **Ý nghĩa:** Gradient chỉ về **hướng dốc LÊN mạnh nhất**. Muốn xuống núi thì đi **ngược** hướng gradient.

```
Nhìn từ trên xuống "bản đồ địa hình" của loss theo (w, b):

b ▲
  │   ┌─────────────┐
  │   │  ┌───────┐  │
  │   │  │  ┌─┐  │  │      ★ = đáy (loss nhỏ nhất)
  │   │  │  │★│  │  │      ● = vị trí hiện tại
  │   │  │  └─┘  │  │      ↗ gradient: hướng leo dốc nhất
  │   │  └───●───┘  │      ↘ -gradient: hướng xuống dốc nhất
  │   └────/↘───────┘
  └──────────────────► w
```

4.3. Chain rule (Quy tắc chuỗi), phiên bản đủ dùng

Loss của ta có dạng "bình phương của một biểu thức chứa w": $(y - w \cdot x)^2$. Để lấy đạo hàm, dùng quy tắc chuỗi:

> **Đạo hàm của hàm lồng nhau = đạo hàm lớp ngoài × đạo hàm lớp trong.**

Ví dụ đời thực: Bạn chạy nhanh gấp 2 lần bạn đi bộ, xe đạp nhanh gấp 3 lần bạn chạy → xe đạp nhanh gấp 2 × 3 = 6 lần đi bộ. Các "tỉ lệ thay đổi" nhân với nhau.

Tính thử: $m(w) = (5 - 2w)^2$. Đặt $u = 5 - 2w$ (lớp trong), $m = u^2$ (lớp ngoài).

- Lớp ngoài: $(u^2)' = 2u$
- Lớp trong: $(5 - 2w)' = -2$
- $m'(w) = 2u \times (-2) = -4(5 - 2w)$. Tại $w = 1$: $-4 \times 3 = \mathbf{-12}$.

```
def m(w):
    return (5 - 2 * w) ** 2
```

```
print(round(dao_ham_so(m, 1.0), 3), -4 * (5 - 2 * 1.0))   # đò số vs công thức
```

Kết quả mẫu:

```
-12.0 -12.0
```

Quy tắc chuỗi chính là nền tảng của **Backpropagation (Lan truyền ngược)**, cách mạng nơ-ron học (sẽ học kỹ ở bài 3.4).

---

5. Gradient Descent

5.1. Công thức cập nhật

```
w_mới = w_cũ − η × ∂L/∂w
```

Ký hiệu	Ý nghĩa
w_cũ	Vị trí hiện tại
∂L/∂w	Độ dốc tại vị trí hiện tại
η (đọc "eta")	**Learning rate (Tốc độ học)**: độ dài mỗi bước
Dấu trừ	Đi **ngược** độ dốc: dốc lên bên phải thì đi sang trái

Vì sao công thức này tự "biết đường"?

- Đạo hàm **âm** (đang ở bên trái đáy, dốc xuống về phía phải) → trừ một số âm = **cộng** → w tăng → đi sang phải. ✓
- Đạo hàm **dương** (bên phải đáy) → w giảm → đi sang trái. ✓
- Càng gần đáy, đạo hàm càng nhỏ → **bước tự động ngắn lại**. ✓

5.2. Tính tay vài bước

Hàm loss đồ chơi: $L(w) = (w - 3)^2$. Đáp án đúng rõ ràng là w = 3. Xem máy có tự tìm ra không.

- Đạo hàm (quy tắc chuỗi): $L'(w) = 2(w - 3)$.
- Bắt đầu w = 0, learning rate η = 0,1.

Bước	w cũ	Gradient 2(w − 3)	w mới = w − 0,1 × gradient
1	0	2(0 − 3) = −6	0 − 0,1 × (−6) = **0,6**
2	0,6	2(0,6 − 3) = −4,8	0,6 + 0,48 = **1,08**
3	1,08	−3,84	**1,464**

```
def L(w):
    return (w - 3) ** 2

def dL(w):
    return 2 * (w - 3)        # đạo hàm tính bằng quy tắc chuỗi

w = 0.0                       # điểm xuất phát
lr = 0.1                      # learning rate
for buoc in range(1, 6):
    g = dL(w)                 # 1. dò độ dốc
    w = w - lr * g            # 2. bước ngược độ dốc
    print(f"Bước {buoc}: gradient = {g:7.3f} → w = {w:.4f}, L = {L(w):.4f}")

for _ in range(45):           # chạy thêm 45 bước nữa
    w = w - lr * dL(w)
print("Sau 50 bước: w =", round(w, 6))
```

Kết quả mẫu:

```
Bước 1: gradient =  -6.000 → w = 0.6000, L = 5.7600
Bước 2: gradient =  -4.800 → w = 1.0800, L = 3.6864
Bước 3: gradient =  -3.840 → w = 1.4640, L = 2.3593
Bước 4: gradient =  -3.072 → w = 1.7712, L = 1.5099
Bước 5: gradient =  -2.458 → w = 2.0170, L = 0.9664
Sau 50 bước: w = 2.999957
```

Máy **tự tìm ra w ≈ 3** mà không ai nói đáp án. Để ý gradient nhỏ dần (−6 → −4,8 → −3,84...) nên bước đi tự chậm lại khi gần đáy.

---

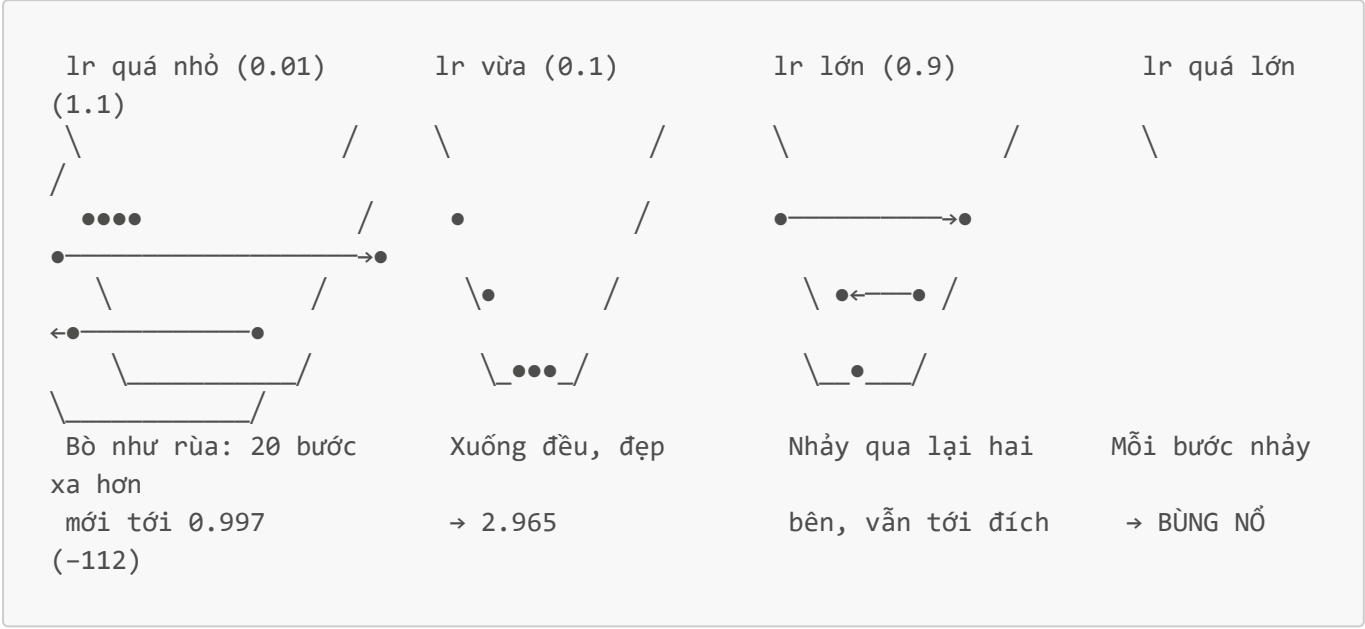
6. Learning rate: bước dài hay ngắn?

```
for lr in [0.01, 0.1, 0.9, 1.1]:
    w = 0.0
    lich_su = []
    for _ in range(20):
        w = w - lr * dL(w)
        lich_su.append(w)
    print(f"lr = {lr}: w sau 5 bước = {[round(v, 2) for v in lich_su[:5]]} ... sau
20 bước = {w:.3f}")
```

Kết quả mẫu:

```
lr = 0.01: w sau 5 bước = [0.06, 0.12, 0.18, 0.23, 0.29] ... sau 20 bước = 0.997
lr = 0.1: w sau 5 bước = [0.6, 1.08, 1.46, 1.77, 2.02] ... sau 20 bước = 2.965
lr = 0.9: w sau 5 bước = [5.4, 1.08, 4.54, 1.77, 3.98] ... sau 20 bước = 2.965
```

```
lr = 1.1: w sau 5 bước = [6.6, -1.32, 8.18, -3.22, 10.46] ... sau 20 bước =
-112.013
```



Learning rate	Hiện tượng	Hậu quả
Quá nhỏ	Đi rất chậm	Tốn thời gian, có thể dừng trước khi tới đáy
Vừa phải	Xuống đều	✅
Lớn	Dao động qua lại	Có thể vẫn tới đích nhưng không ổn định
Quá lớn	Càng đi càng xa	**Phân kỳ (diverge)**: loss tăng vọt, có khi ra nan

Chọn learning rate là một trong những việc **quan trọng nhất** khi huấn luyện mô hình (sẽ học thêm các cách chọn thông minh ở bài 3.5). Giá trị khởi đầu phổ biến: 0,1; 0,01; 0,001.

---

7. Tự viết mô hình học từ dữ liệu

7.1. Tính gradient của MSE

Với $\hat{y}_i = w \cdot x_i + b$ và $L = (1/n)\ \Sigma(\hat{y}_i - y_i)^2$, dùng quy tắc chuỗi:

$$\partial L/\partial w = (2/n)\ \Sigma\ (\hat{y}_i - y_i) \cdot x_i$$
$$\partial L/\partial b = (2/n)\ \Sigma\ (\hat{y}_i - y_i)$$

Phần	Đến từ đâu
$2(\hat{y}_i - y_i)$	Lớp ngoài: đạo hàm của $(\ldots)^2$
$\times\ x_i$	Lớp trong: đạo hàm của $(w \cdot x_i + b)$ theo w là x_i
$\times$ 1 (với b)	Đạo hàm của $(w \cdot x_i + b)$ theo b là 1

Đọc bằng lời: Nếu mô hình đang đoán **quá cao** (ŷ − y dương) → gradient dương → giảm w, b. Nếu đoán **quá thấp** → tăng. Căn có x lớn ảnh hưởng tới w nhiều hơn.

Tính tay bước đầu tiên (w = 0, b = 0, sai số = ŷ − y = [−2; −2,6; −2,9; −3,6; −4]):

- $\partial L/\partial w = (2/5) \times (-2\times4 - 2{,}6\times5 - 2{,}9\times6 - 3{,}6\times7 - 4\times8) = 0{,}4 \times (-8 - 13 - 17{,}4 - 25{,}2 - 32) = 0{,}4 \times (-95{,}6) =$ **−38,24**
- $\partial L/\partial b = (2/5) \times (-15{,}1) =$ **−6,04**
- Với η = 0,01: w = 0 − 0,01 × (−38,24) = **0,3824**; b = 0 + 0,0604 = **0,0604**.

7.2. Code

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([4, 5, 6, 7, 8], dtype=float)      # diện tích (chục m²)
y = np.array([2.0, 2.6, 2.9, 3.6, 4.0])         # giá thật (tỷ)

w, b = 0.0, 0.0          # khởi tạo tham số
lr = 0.01                # learning rate
lich_su_loss = []        # ghi lại loss để vẽ

for epoch in range(2001):                        # epoch = một lần duyệt hết dữ
liệu (bài 3.6)
    y_du_doan = w * x + b                        # 1. dự đoán cho cả 5 căn (vector
hóa)
    sai_so = y_du_doan - y                       # 2. sai số từng căn
    loss = (sai_so ** 2).mean()                  # 3. MSE
    lich_su_loss.append(loss)

    dw = 2 * (sai_so * x).mean()                 # 4. gradient theo w
    db = 2 * sai_so.mean()                       #    gradient theo b

    w = w - lr * dw                              # 5. bước xuống dốc
    b = b - lr * db

    if epoch in [0, 1, 2, 10, 100, 500, 1000, 2000]:
        print(f"Epoch {epoch:>4}: loss = {loss:.5f} | w = {w:.4f} | b = {b:.4f}")

print("Dự đoán căn 65 m²:", round(w * 6.5 + b, 3))
```

Kết quả mẫu (loss in ra là loss **trước** khi cập nhật, w và b là giá trị **sau** khi cập nhật):

```
Epoch    0: loss = 9.62600 | w = 0.3824 | b = 0.0604
Epoch    1: loss = 0.47575 | w = 0.4669 | b = 0.0737
Epoch    2: loss = 0.02873 | w = 0.4856 | b = 0.0766
Epoch   10: loss = 0.00577 | w = 0.4910 | b = 0.0770
Epoch  100: loss = 0.00574 | w = 0.4918 | b = 0.0719
Epoch  500: loss = 0.00566 | w = 0.4946 | b = 0.0544
Epoch 1000: loss = 0.00562 | w = 0.4967 | b = 0.0406
```

```
Epoch 2000: loss = 0.00560 | w = 0.4988 | b = 0.0274
Dự đoán căn 65 m²: 3.27
```

Epoch 0 khớp với tính tay: loss = 9,626; w = 0,3824; b = 0,0604. ✓

Đọc kết quả:

- Loss giảm từ 9,6 xuống 0,0056 chỉ sau vài chục bước. Máy **đã học**!
- w ≈ 0,5: mỗi chục m^2 thêm khoảng 0,5 tỷ (tức 0,05 tỷ/m^2, khớp bài 1.7).
- Đáp án chính xác (tính bằng công thức toán) là w = 0,5; b = 0,02. Để ý **w gần đúng rất nhanh, nhưng b nhích cực chậm** (0,077 → 0,027 sau 2.000 epoch). Nguyên nhân: x có giá trị từ 4 đến 8, không nằm quanh 0, nên "thung lũng" loss bị kéo dài và hẹp theo một hướng. **Chuẩn hóa dữ liệu** (bài 1.11) sẽ sửa được vấn đề này.

7.3. Vẽ đường cong loss

```
plt.plot(lich_su_loss)
plt.yscale("log")            # thang log để thấy rõ cả đoạn giảm nhanh lẫn đoạn giảm chậm
plt.xlabel("Epoch"); plt.ylabel("Loss (MSE)")
plt.title("Loss giảm dần qua quá trình học")
plt.show()
```

Bạn sẽ thấy: loss rơi thẳng đứng trong ~5 epoch đầu, sau đó gần như nằm ngang. Bạn sẽ vẽ biểu đồ này **cho mọi mô hình** ở Giai đoạn 3 để theo dõi quá trình học.

> 🎉 Bạn vừa tự viết từ con số 0 thuật toán học đứng sau Linear Regression. Ở bài 2.2, Scikit-learn làm việc này trong 2 dòng. Nhưng giờ bạn biết **bên trong nó làm gì**.

---

8. Bài tập

1. Cho $L(w) = (w + 1)^2 + 2$. Đáp án đúng là w bằng bao nhiêu? Bắt đầu từ w = 4, η = 0,25, **tính tay** 3 bước Gradient Descent.
2. Dùng hàm `dao_ham_so` ở mục 3.2 tính đạo hàm của $k(w) = w^3 - 3w$ tại w = 2. So sánh với công thức (quy tắc ở mục 3.3).
3. Trong code mục 7.2, đổi `lr = 0.05`. Chuyện gì xảy ra? Thử `lr = 0.03` và `lr = 0.02`.
4. Trong code mục 7.2, đổi x sang đơn vị m^2 thật (`x = np.array([40, 50, 60, 70, 80])`) và giữ `lr = 0.01`. Chuyện gì xảy ra? Vì sao? Phải đổi lr thành khoảng bao nhiêu để chạy được?

Đáp án

> Mẹo Notion: chọn tiêu đề "Đáp án" → `Turn into → Toggle heading 3`.

1. Đáp án đúng: w = −1 (chỗ $(w + 1)^2 = 0$). Đạo hàm: $2(w + 1)$.
    - Bước 1: w = 4 − 0,25 × 2 × 5 = 4 − 2,5 = **1,5**
    - Bước 2: w = 1,5 − 0,25 × 2 × 2,5 = 1,5 − 1,25 = **0,25**

- Bước 3: w = 0,25 − 0,25 × 2 × 1,25 = 0,25 − 0,625 = **−0,375**
- Mỗi bước giảm một nửa khoảng cách tới −1. Hằng số "+2" không ảnh hưởng đáp án vì đạo hàm của nó bằng 0.

2. `dao_ham_so(k, 2)` ≈ **9,0**. Công thức: k'(w) = $3w^2 - 3$ → 3×4 − 3 = **9**. ✓

3. Với `lr = 0.05` và `lr = 0.03`: loss tăng vọt rồi thành `inf` hoặc `nan` → **phân kỳ**. Với `lr = 0.02`: hội tụ. Với dữ liệu này, ngưỡng an toàn là khoảng **0,0257**; vượt qua là nổ. Ngưỡng đó phụ thuộc vào độ lớn của x.

4. Với x lớn gấp 10 lần, gradient theo w lớn gấp ~10 lần, và "độ cong" của thung lũng lớn gấp ~100 lần → `lr = 0.01` (thậm chí `0.001`) quá lớn, mô hình **nổ** ra `nan`. Ngưỡng an toàn giảm còn khoảng 0,00026; phải dùng cỡ `lr = 0.0001` mới chạy được (sau 2.000 epoch: w ≈ 0,0503, b ≈ 0,0012, b gần như không nhúc nhích). Bài học: **đơn vị/độ lớn của dữ liệu ảnh hưởng trực tiếp đến việc học**. Đây là lý do gần như luôn phải **chuẩn hóa** dữ liệu trước khi huấn luyện (bài 1.11).

---

9. Flashcard

Mặt trước	Mặt sau
Đạo hàm cho biết gì?	Độ dốc: x tăng một chút thì f thay đổi bao nhiêu, theo hướng nào.
Đạo hàm = 0 nghĩa là?	Chỗ phẳng, có thể là đáy (điểm nhỏ nhất).
Đạo hàm riêng $\partial L/\partial w$?	Độ dốc theo w khi giữ các biến khác cố định.
Gradient là gì?	Vector các đạo hàm riêng, chỉ hướng dốc lên mạnh nhất.
Quy tắc chuỗi?	Đạo hàm hàm lồng nhau = đạo hàm ngoài × đạo hàm trong.
Công thức Gradient Descent?	$w \leftarrow w - \eta \times \partial L/\partial w$
Vì sao có dấu trừ?	Để đi ngược hướng dốc lên, tức là đi xuống.
Learning rate quá lớn thì sao?	Phân kỳ: loss tăng vọt, ra nan/inf.
Learning rate quá nhỏ thì sao?	Học rất chậm.
Loss là gì?	Con số đo mức độ sai của mô hình. Học = làm loss nhỏ đi.

Bài tiếp theo: 1.10 – Toán: xác suất cơ bản.